

Lab Assignment 5

(Due: June 12)

1. Use openssl to generate RSA public/private key

We can generate RSA private key (p, q, d) using openssl:

```
$ openssl genrsa -aes128 -out private.pem 1024
```

This will generate a rsa instance (p, q, d, e, n) with p, q of 1024 bits and to prevent leaking the private key, the output private.pem is encrypted by aes128 cipher with password you will be prompted to provide. Now use the above command to generate a rsa private key and save it in file private.pem. Then, extract the public key (e, n) in a file public.pem:

```
$ openssl rsa -in private.pem -pubout >public.pem
```

You can display private key using

```
$openssl rsa -in private.pem -text -noout
```

You also can display public key using

```
$openssl rsa -in public.pem -pubin -text -noout
```

Take screen for the displays for these two files, as evidence of your work.

2. In this problem, you need to practice RSA encryption and decryption.

a. Encrypt messages using PKCS1_OAEP, which is an implementation of RSA. Use the key **RsaKey** derived above to do the encryption. The functions are described as follow.

- **Cipher=PKCS1_OAEP.new(RsaKey):**
 - For the encryption, RsaKey is a public-key. Return an encryption object **Cipher**.
- **Cipher.encrypt(message):**
 - This returns ciphertext of message (byte string) under encryption object **Cipher**.

Encrypt message='your name and ID' and save ciphertext into a file. Take a screen shot for hexdump of your ciphertext (**\$hexdump -C filename**). Ref. **encrypt_RSA.py**.

b. Decrypt the ciphertext in (a). The functions are described as follow.

- **Cipher=PKCS1_OAEP.new(RsaKey):**
 - For the decryption, RsaKey is a private-key. Return an decryption object **Cipher**.
- **Cipher.decrypt(ctxt):**
 - This returns message='your name and ID' under decryption object **Cipher**.

Take a screen shot for your decryption. Ref. **decrypt_RSA.py**.

3. (optional) In this problem, you practice RSA signature: generation and verification.

a. Generate RSA based signature. The functions are described as follows.

- `Signer=pss.new(RsaKey):`
 - This defines a signing object *signer* with *RsaKey* (imported from your RSA private key file).
- `Signer.sign(hashmessage):`
 - This generates the RSA signature of the hashed message. Here you can use SHA512 to generate the hash value of your message.

M = "I owe you \$2000". Change \$2000 to \$3000 and sign the modified message. Compare both signatures. Are they similar? Save your signature into a file. Take a screen shot for your file content (using hexdump). Ref. **sign_RSA.py**

b. Verify the signature in (a). The functions are described as follows.

- `Signer=pss.new(RsaKey):`
 - This defines a signing object *signer* with *RsaKey* (imported from your RSA public key file).
- `Signer.verify(hashmessage, signature):`
 - This verifies if *signature* is consistent with the *hashed message*.

Take a screen shot for the output result. Ref. **verify_RSA.py**

4. In this problem, you will use Diffie-Hellman with authentication to protect the client-server communication. Implement the following functionalities.

a. Create two files: TCP client and TCP server, capable to chat with each other using socket.

b. Client and Server execute Diffie-Hellman to generate a shared key and use sha256 to hash this shared key to 32-byte secret **sk**. Diffie-Hellman uses parameters:

```
p=2582249878086908589655919172003011874329705792829223512830659356540647622016841194629645353280137831435903171972747559779
g=2
```

Note: *x, y* in Diffie-Hellman can be obtained with **`Crypto.Random.random.getrandbits(400);`**

see <https://pycryptodome.readthedocs.io/en/latest/src/random/random.html> if necessary.

c. Sender (Client or Server) uses **sk** as a secret key of AES to encrypt your chat message in (a).

This results in ciphertext *C* and computes *tag=sha256(C)*. In (a), sender sends (*C, tag*), instead of plain chat message.

d. At the receiver, when receiving (*C, tag*), verify whether *tag=sha256(C)* holds. If it fails, raise exception; otherwise, use *sk* as the AES secret to decrypt *C*. This will recover your chat message.

Paste your client and server programs in your submission file. Print out *sk, C, tag* and decrypted *chat message* in (d) for one *chat message*.